System Verification and Validation Plan for LingoLyrics

Team #21, LingoLyrics
Rhea Gokhale
Aansh Kotian
Omar Bakr
Raymond Ma

November 5, 2026

Revision History

Table 1: Revision History

Date	Developer(s)	Change
October 27, 2026	Aansh Kotian	Initial Revision 0 draft

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 3
- 2.4 Relevant Documentation . . . 3

3 Plan **4**
- 3.1 Verification and Validation Team . . . 4
- 3.2 SRS Verification . . . 5
- 3.3 Design Verification . . . 6
- 3.4 Verification and Validation Plan Verification . . . 7
- 3.5 Implementation Verification . . . 7
- 3.6 Automated Testing and Verification Tools . . . 8
- 3.7 Software Validation . . . 9

4 System Tests **9**
- 4.1 Tests for Functional Requirements . . . 10
  - 4.1.1 Lesson Import and Validation . . . 10
  - 4.1.2 Playback and Synchronization . . . 12
  - 4.1.3 Lesson Display and Content . . . 15
  - 4.1.4 Anki Handoff . . . 18
  - 4.1.5 Editor . . . 20
- 4.2 Tests for Nonfunctional Requirements . . . 23
  - 4.2.1 Performance . . . 23
  - 4.2.2 Accessibility and Appearance . . . 24
  - 4.2.3 Capacity . . . 25
  - 4.2.4 Compatibility, Installability, and Privacy . . . 25
  - 4.2.5 Usability Studies . . . 26
  - 4.2.6 Static Tests . . . 27
- 4.3 Traceability Between Test Cases and Requirements . . . 30

5 Unit Test Description **33**
- 5.1 Unit Testing Scope . . . 33
- 5.2 Tests for Functional Requirements . . . 33

5.3 Tests for Nonfunctional Requirements . . . 33
5.4 Traceability Between Test Cases and Modules . . . 34

6 Appendix 35
6.1 Symbolic Parameters . . . 35
6.2 Usability Tasks and Survey Questions . . . 36

7 Appendix — Generative AI Use Disclosure 38
7.1 AI Tools Used . . . 38
7.2 How and Where Used . . . 38
7.3 Verification of AI-Generated Content . . . 38

8 Appendix — Reflection 39

List of Tables

1 Revision History . . . i
2 Verification and validation responsibilities . . . 5
3 Automated testing and verification tools . . . 8
4 Static tests . . . 28
5 Tests and the requirements they support . . . 31
6 Symbolic parameters used by the tests . . . 35

List of Figures

1 Symbols, Abbreviations, and Acronyms

Requirement identifiers (for example FR4, HS2, IN8) and the symbolic constants (for example `SYNC_TOL`) are defined in the SRS (Team LingoLyrics, 2026). The notation used for test identifiers and the terms used in the tests are listed below.

symbol	**description**
T	Test
FT-n	Functional system test number n (Section 4.1)
NT-n	Nonfunctional system test number n (Section 4.2)
ST-n	Static test (review or inspection) number n (Section 4.2.6)
G1 to G6	Project goals defined in the Problem Statement and the SRS
R1 to R3	Proof of Concept (PoC) risks defined in the Development Plan
CI	Continuous integration (GitHub Actions)
PoC	Proof of Concept
SRS	Software Requirements Specification
V&V	Verification and validation
MG, MIS	Module Guide and Module Interface Specification (design documents)
LUFS, LU	Loudness units used by requirement HS9

term	meaning in this plan
Song map	Structured lesson file that connects an audio track with timed lyrics, translations, transliterations, annotations, and language metadata
Acceptance suite	The fixed two-song suite containing one song map for each demonstrated language, defined before implementation testing begins
Annotated target	A word or line with a translation and a contextual annotation
Complete map	A song map containing all required audio, timing, lyric, and annotation data
Test creator	A participant given the same supplied audio, lyric, and annotation material, the creator guide, and no other help
Test track	The pre-defined short track of at most `TEST_TRACK_LINES` lyric lines used in the creator study
Invalid map set	A set of song maps prepared for testing, with at least one map that breaks each validity condition V1 to V8 of the SRS, plus malformed maps (truncated, wrong types, oversized, deeply nested)
Test deck	The Anki deck named *LingoLyrics Test*

This document describes how the team will verify and validate LingoLyrics. It states what is being tested and why, who is responsible, how each document and the code will be checked, and the system tests that show whether the goals and requirements in the SRS are met. Section 2 gives general information. Section 3 describes the verification and validation plan. Section 4 defines the system tests and traces them to the requirements. Section 5 will describe the unit tests once the detailed design is complete. The appendices list the symbolic constants used by the tests, the tasks for the usability studies, and the generative AI use disclosure.

2 General Information

2.1 Summary

The software being tested is LingoLyrics, a song-based language-learning product. It has three parts:

- a **learner app** for iOS and Android that plays song lessons with synchronized lyric highlighting, shows translations, transliterations, and annotations for selected words, and sends selected vocabulary to Anki;
- a web-based **song map editor** in which creators time lyrics, review word segmentation and readings, enter translations and annotations, preview a lesson, and export a validated song map; and
- a **shared core** containing the versioned JSON Schema, the validation code, and the shared TypeScript types used by both.

2.2 Objectives

The objectives of the verification and validation effort are:

- **Show that the six goals are met.** Goals G1 to G6 each have a measurable criterion that is evaluated on the acceptance suite (or in the creator study for G4). Showing that these criteria are met is the main objective.

- **Build confidence in the correctness of song map handling.** Both the editor and the learner app must accept valid maps and reject invalid, malformed, or hostile maps, because a lesson cannot be correct if the map is not.
- **Build confidence that the safety and security requirements work.** The requirements that came from the Hazard Analysis (HS, AC, IN, PR, AU, IM, and LG) are tested because they protect the learner's hearing, data, and Anki collection.
- **Demonstrate adequate usability** for the two user groups: creators must be able to produce a lesson without editing the map file by hand, and a first-time learner must be able to study a lesson and send a word to Anki without help.
- **Resolve the three PoC risks early** (R1 lyric synchronization on a phone, R2 segmentation of lyrics, R3 sending words to Anki from a phone) so that the later plan rests on measured results.

The following are out of scope for verification and validation:

- **Software from other vendors.** The operating systems, browsers, Anki, AnkiDroid, and AnkiConnect are assumed to behave as documented (assumption A4 in the SRS). The same assumption is made for the libraries the project uses, such as Expo Audio, Howler.js, kuromoji.js, and the JSON Schema validator. The team tests how the product uses them (for example, segmentation accuracy and highlight timing), not the libraries themselves. Library versions are pinned and checked for known vulnerabilities (IM3).
- **Stretch goals.** A third language, collaborative review, learner progress and further accessibility features, and automatic timing assistance are not required for the product to be viable and are not scheduled.
- **Content outside the initial release.** The initial release uses only team-created, public-domain, or licensed content and does not include streaming-service integration, a production music catalogue, or universal language support, so none of these are tested.

- **Store publication.** The learner app is installed as an Expo development or preview build and store publication is not required (OE4), so app store review is not tested.
- **User accounts, a server, and cloud sync.** The initial release has none of these.

These items are left out because the team has limited time and people, and because the items either are outside the control of the team or are not part of the minimum viable product.

2.3 Custom Documents

The project has two custom documents, taken from the Problem Statement:

- **Fall: Design Thinking Report or User/Stakeholder/Client Interview Report.** Interviews with target learners and song map creators validate the problem and the learner and creator workflows before design begins.
- **Winter: Usability Report.** Usability testing of the learner app and the editor with representative users, measured against the goals. This replaced the User Manual after instructor feedback.

Both are part of validation (Section 3.7). The Usability Report presents the results of tests NT-10 and NT-11.

2.4 Relevant Documentation

The following documents are used by this plan.

- **SRS** (Team LingoLyrics, 2026): the source of every requirement tested here. Each requirement has a fit criterion that says how it will be tested, and the SRS contains the symbolic constants, the formal definition of a valid song map, and traceability tables. The tests in Section 4 are written from those fit criteria.
- **Problem Statement and Goals**: defines the goals G1 to G6, the acceptance suite, the test creators, and the custom documents.

- **Development Plan**: defines the PoC plan with its pass criteria, the continuous integration checks, the tools, the review rules, the team roles, and the schedule.
- **Hazard Analysis**: the safety and security requirements share their identifiers with the SRS, so each hazard can be traced to a requirement and then to a test.
- **Module Guide and Module Interface Specification** (due November 16, 2026 and January 18, 2027): needed before the unit tests in Section 5 can be written.

3 Plan

This section describes who carries out verification and validation and how each artifact is checked. Sections 3.1 to 3.3 cover the documents, Sections 3.5 and 3.6 cover the implementation, and Section 3.7 covers validation with users and the supervisor. The plan follows the course schedule: this V&V Plan is due November 2, 2026, the PoC demonstration takes place between November 23 and December 4, 2026, the Revision 0 demonstration between February 1 and 12, 2027, the V&V Report on March 8, 2027, and the final demonstration between March 15 and 22, 2027.

3.1 Verification and Validation Team

All team members create issues, develop, test, document, and review code. Table 2 gives each person's main responsibility for verification. Component owners are taken from the Development Plan.

Roles that apply to everyone:

- Every pull request is reviewed by a member who does not own that component, and each component has a backup member who reviews its pull requests.
- The Content and QA Manager ticks off the deliverable checklists and reviews the release pull request before each deliverable. The administrative roles rotate after each major deliverable, so no member reviews their own work every time.
- Each component owner demonstrates the PoC risk for their component.

Table 2: Verification and validation responsibilities

Person	**Component owned**	**Verification responsibility**
Omar Bakr	Learner app: playback and lyric sync	Maintains the tests for playback and synchronization and for lesson display (Sections 4.1.2 and 4.1.3); runs the R1 part of the PoC.
Raymond Ma	Song map editor and schema	Maintains the tests for lesson validation and for the editor (Sections 4.1.1 and 4.1.5); owns the shared validator and its unit tests.
Rhea Gokhale	Language processing (segmentation, romanization)	Maintains the segmentation, rendering, and romanization tests (FT-18, FT-19, FT-32); runs the R2 part of the PoC.
Aansh Kotian	Anki integration and CI/CD	Maintains the Anki tests (Section 4.1.4) and the CI pipeline and coverage reporting; runs the R3 part of the PoC.
Supervisor, Dr. Angela Zavaleta Bernuy	—	Acts as the client: reviews the scope and goals and accepts the deliverables at each milestone.
TA and the assigned peer review team	—	Review the documents through the informal presentation and GitHub issues.

3.2 SRS Verification

The SRS will be verified with the following approaches.

- **Peer review.** The assigned peer review team reads the SRS and raises GitHub issues with the Peer-Review label. Each issue is answered, and the SRS is revised or the reason for not changing it is recorded in the issue.
- **Informal presentation with the TA.** The team presents the project and the SRS, and the TA's feedback is recorded as issues.

- **Supervisor review.** The team presents the goals, the personas, and the open issues listed in the SRS at a supervisor meeting (Fridays, 10:00 to 10:30 on Microsoft Teams) and asks the supervisor to decide the open questions that affect scope: the second language, how lessons reach the learner app, and the privacy obligations. The supervisor's decisions are recorded in the supervisor meeting issue.

- **Checklist inspection.** Before each revision the Content and QA Manager checks the SRS against the course checklist and rubric, and against the following SRS checklist:

  - every requirement has a unique identifier, a rationale for every requirement that needs one, a fit criterion, and *Traces to* and *Referenced by* entries;
  - every fit criterion can be tested, and every number is a symbolic constant defined only in the SRS appendix with a source or reason;
  - every goal, use case, and Hazard Analysis row appears in the traceability tables, and each of them has at least one requirement;
  - requirements describe what the product does, not how it is built, and design decisions that are imposed are listed as constraints;
  - the likely and unlikely changes are listed, and the assumptions are stated.

- **Test-derived review.** Writing the tests in Section 4 from the fit criteria checks that each requirement is verifiable. A requirement for which no test can be written is raised as an issue against the SRS.

3.3 Design Verification

The Module Guide and Module Interface Specification will be verified by:

- review by the classmates assigned as peer reviewers, the TA, and the supervisor, with issues created for each finding;

- a team review by someone who did not write the module, using a design checklist; and

- a check against the SRS that confirms the following, each of which comes from a requirement: the editor and the learner app use the same validation module and schema version (FR16); the learner app contains no tokenizer and displays stored segmentation (IN4); the schema's language-specific fields are optional and keyed by language code (SC1); and every requirement is assigned to at least one module.

3.4 Verification and Validation Plan Verification

This plan is verified by:

- review by the classmates assigned as peer reviewers and by the TA, with issues created for each finding;
- a team review using the course checklist for the V&V Plan; and
- a coverage check using Table 5: every requirement that has a fit criterion appears in at least one test, and every test refers to at least one requirement. A requirement that does not appear is either given a test or recorded with a reason.

3.5 Implementation Verification

The implementation is verified in four ways.

- **System tests.** The tests in Section 4 are run on the release build. The unit tests will be listed in Section 5.
- **Static analysis.** ESLint and Prettier check every commit (through Husky hooks) and every pull request against the Airbnb JavaScript/TypeScript Style Guide, and the TypeScript compiler checks types.
- **Code review.** Every pull request needs an approving review by a member who does not own the component. The pull request template asks the author to confirm that tests and lint pass, to link the issue, and to list any AI-assisted changes. Changes to dependencies need approval from a second member (IM3), and every call to Anki is reviewed (AC1).
- **Code review interviews.** The PoC and final demonstrations include code review interviews, so each member must be able to explain the code they submit, including AI-assisted code.

Reviews and inspections of the product and its documentation are listed in Section 4.2.6.

3.6 Automated Testing and Verification Tools

The tools are described in the Development Plan. Table 3 summarizes how they are used for verification.

Table 3: Automated testing and verification tools

Tool	**Use**
GitHub Actions	Runs on every pull request and on pushes to `main`. A pull request cannot be merged into `develop` or `main` unless all checks pass.
Jest	Unit and integration tests, with a coverage report in CI. The shared core must have statement coverage of at least `COVERAGE` (MS2); the coverage report is read from the CI run.
ESLint, Prettier, Husky	Enforce the coding standard before commit and in CI.
JSON Schema and the shared validator	CI validates every example song map in the repository on each pull request (MS1, FR16).
Dependency lock files and an automated vulnerability check	Versions are pinned, and CI fails when a high or critical vulnerability is reported (IM3).
TypeDoc	Generates API documentation from TSDoc comments.
LaTeX build in CI	Builds the documents to PDF so the submitted documents match the source.

The details of this section will change as the implementation grows.

3.7 Software Validation

Validation checks that the requirements describe the right product. It uses:

- **Stakeholder interviews (fall custom document).** Interviews with target learners and creators check the problem and the learner and creator workflows before design begins. Findings that change a requirement are raised as issues against the SRS.
- **The PoC demonstration.** The three PoC checks (R1 to R3) show on a real phone whether the technical approach works. Their results confirm or change the symbolic constants marked *initial* in the SRS and decide the likely changes LC1 and LC3.
- **Supervisor review.** The supervisor, who acts as the client, reviews and accepts the deliverables at each milestone, including the Revision 0 demonstration. The team presents the working product and records the supervisor's feedback as issues.
- **Usability studies (winter custom document).** Tests NT-10 and NT-11 measure goal G4 and the learner usability requirements with representative users, using only the written instructions. Requirements UH1, UH2, and UH5 are built by Revision 0 but confirmed after these studies.

There is no external data set for validation. The acceptance suite is made of team-created, public-domain, or licensed songs and is defined before implementation testing begins.

4 System Tests

The system tests are written from the fit criteria of the SRS and are grouped by area. Tests marked *Automatic* run in CI or from a script. Tests marked *Manual* are carried out by a team member who records the results in the test issue.

Test records. Each execution records the test ID, tester, date, build commit, device or browser version, and the version or commit of the input files. The linked test issue includes the expected result, actual result, pass/fail status, and supporting logs or screenshots. A failed test links to

a defect issue. After a fix, the affected test is rerun and both results are retained. Tests that cannot be executed are marked blocked with a reason and are not counted as passes.

Test environment. Unless a test says otherwise, tests run on one iOS device and one Android device with the same release build of the learner app (OE4, MS4), and the editor runs in each supported browser. The acceptance suite has one song map for each of the `LANG_COUNT` demonstrated languages. Japanese is the language with a non-Latin writing system. The second language has not been chosen yet (SRS, Open Issues), so tests refer to it as "the second acceptance language". A test deck named *LingoLyrics Test* exists in Anki. Symbolic constants are defined in Appendix 6.1.

PoC checks. Three tests are also the PoC pass criteria of the Development Plan. R1 is FT-8 on the test phone. R2 is FT-32 and NT-1 for the Japanese test song. R3 is FT-23 using at least one candidate method.

4.1 Tests for Functional Requirements

The functional tests cover the product use cases PUC1 to PUC4 and the safety and security requirements that change what the product does. They are divided into five areas: lesson import and validation, playback and synchronization, lesson display and content, the Anki handoff, and the editor.

4.1.1 Lesson Import and Validation

These tests cover opening lessons (FR1, FR2) and the checks that decide whether a map is valid (IN2, IN3, IN11, IM1, IM2, FR16, RB1). The validity conditions V1 to V8 are defined in the SRS (Formal Specification).

FT-1 Open valid lessons

Control: Manual.

Initial State: The learner app is installed with no lessons.

Input: Open each song map package in the acceptance suite from device storage.

Output: Every lesson opens. The lesson list shows each lesson's title, artist, language, difficulty, and explicit content flag, and all of them match the song map metadata.

Test Case Derivation: Fit criteria of FR1 and FR2.

How test will be performed: A team member opens each package and compares the listed fields with the map's metadata.

FT-2 Reject invalid maps

Control: Automatic.

Initial State: The shared validator, the editor, and the learner app are built from the same commit.

Input: Each map in the invalid map set that breaks one of V1 to V8, given to the validator, to the editor's export, and to the learner app's import.

Output: Each map is rejected, and the message names the rule that was broken (for example, overlapping lines for V4).

Test Case Derivation: IN2 requires one invalid test map per condition to be rejected by both the editor and the app. FR1 and FR14 require the message to name the failed rule.

How test will be performed: A Jest test runs the validator on every map in the set and compares the reported rules with the expected rules. A manual run repeats the set through the editor and the app.

FT-3 Unsupported schema version

Control: Automatic.

Initial State: The app supports the major versions in `SUPPORTED_MAJOR`.

Input: A map whose major version is not in `SUPPORTED_MAJOR`.

Output: The app refuses to open the map and tells the learner why. Maps that differ only in minor or patch version open.

Test Case Derivation: IN3 and V1.

How test will be performed: Jest test on the validator, and one manual import in the app.

FT-4 Audio file checks on import

Control: Manual.

Initial State: The app is open with no lesson loaded.

Input: Packages whose audio file is missing, cannot be decoded, or has a duration that differs from `audioDuration` by more than `AUDIO_DUR_TOL`.

Output: Each package is rejected with a message, and no lesson is opened.

Test Case Derivation: IN11 and V8.

How test will be performed: A team member imports each package on both devices.

FT-5 Malformed and hostile maps

Control: Automatic (parser), Manual (device).

Initial State: The editor and the app are running.
Input: (a) A truncated map, a map with wrong types, a map larger than `MAX_MAP_SIZE`, and a map nested deeper than `MAX_NESTING`. (b) A map whose text contains HTML or script. (c) Maps whose audio path is absolute, is a URL, or contains a `..` segment. (d) A package with a path that leaves the package.
Output: (a) Each map is rejected without a crash, and oversized and deeply nested maps are rejected before full parsing. (b) The text is shown as plain text. (c) Each map is rejected. (d) No file outside a new folder owned by the app is changed.
Test Case Derivation: Fit criteria of RB1, IM1, and IM2.
How test will be performed: Jest tests for the parser, then a manual import of the test packages on both devices, checking the app's storage before and after (d).

FT-6 Shared validator agreement

Control: Automatic.
Initial State: CI is running on a pull request.
Input: Every example song map in the repository, valid and invalid.
Output: The editor and the learner app use one validator module and give the same verdict for every map. Valid maps open in the app without manual changes.
Test Case Derivation: FR16 requires a single shared module, and G3 requires that an editor map opens in the app.
How test will be performed: Each example map has an expected verdict and expected failed rules derived from the SRS. CI compares the shared validator's results with these expectations. The same maps are passed through the editor's export validation and the app's import validation to confirm that both use the validator correctly. A valid map exported by the editor must open in the app without manual changes.

4.1.2 Playback and Synchronization

These tests cover playback control, lyric highlighting, and the playback safety requirements (FR3, FR4, FR5, IN1, HS1, HS2, HS3, HS8, RB2). The mode transitions are defined in the SRS (Playback Modes).

FT-7 Playback controls

Control: Manual.

Initial State: A lesson from the acceptance suite is open and paused.
Input: Press play, pause, seek to a chosen time, and restart the current line.
Output: After each control, the reported audio position equals the expected position within `SYNC_TOL`.
Test Case Derivation: Fit criterion of FR3.
How test will be performed: The app logs the audio position after each control, and the tester compares it with the expected position.

FT-8 Line and word synchronization

Control: Manual (run), Automatic (analysis).
Initial State: Each acceptance suite lesson is open on the test phone.
Input: Play the lesson from start to end.
Output: At least `SYNC_PASS` of line transitions occur within `SYNC_TOL` of their mapped start times, where a transition is on time when $|h_i - s_i| \leq$ `SYNC_TOL`. For lines with word timings, at least `SYNC_PASS` of word highlights are on time by the same measure. The relative error is reported for each song.
Test Case Derivation: FR4, FR5, PA2, and the definition of the active line in the SRS. The thresholds are the G1 criteria.
How test will be performed: The app logs the playback position at each highlight change, and a script compares the log with the map and reports the percentage that are on time. For uninterrupted playback, the denominator is the number of expected line starts in the map, not the number of logged changes. A missing highlight counts as a failure, and repeated highlights cannot increase the number of successful transitions. Word highlights are evaluated separately using the expected word starts where word timings are provided.

FT-9 Realignment after interruptions

Control: Manual.
Initial State: A lesson is playing.
Input: Seek, pause, resume, and send the app to the background and return to it.
Output: The highlighted line is the active line for the current position after each action, and the pass rate of FT-8 still holds when it is repeated with these actions.
Test Case Derivation: IN1 requires the highlight to be driven by the

audio player's position and realigned after each of these actions.
How test will be performed: Run FT-8 with a script of actions on both devices.

FT-10 No unprompted audio

Control: Manual.
Initial State: The app is closed, then open on the home screen, then in a lesson.
Input: Open the app, open a lesson, resume the app from the background, and return to a lesson.
Output: Audio does not start until the learner presses play.
Test Case Derivation: HS1 and the mode table, which allows no transition to Playing except a learner action.
How test will be performed: A tester performs each action with the volume up and notes whether audio starts.

FT-11 Pause on audio output disconnect

Control: Manual.
Initial State: A lesson is playing through wired headphones, then through Bluetooth headphones.
Input: Unplug the wired headphones, then disconnect the Bluetooth headphones.
Output: Playback pauses within `DISCONNECT_PAUSE` and no audio plays from the phone speaker.
Test Case Derivation: HS2 and the transition from Playing on audio output disconnection.
How test will be performed: Repeat on iOS and Android with a stopwatch or screen recording to measure the time to pause.

FT-12 Fade in at start and resume

Control: Manual (recording), Automatic (analysis).
Initial State: A lesson is paused with the volume at a set level.
Input: Press play, then pause and press play again.
Output: In a recording of the app's audio output, the volume rises from silence to the set level over `FADE_IN`, within 0.1 s, at each start and resume.
Test Case Derivation: Fit criterion of HS8.
How test will be performed: Record the output on iOS and Android and measure the rise time from the waveform.

FT-13 Playback with the screen locked

Control: Manual.

Initial State: The app has just been installed, and a full acceptance suite lesson is ready.

Input: Launch the app for the first time, press play, and lock the screen. Later, launch the app a second time.

Output: The lesson plays to the end with the screen locked. The notice advising against using the screen while driving, cycling, or crossing traffic appears on the first launch after installation and not on the second.

Test Case Derivation: Fit criterion of HS3.

How test will be performed: Run on iOS and Android and listen to the end of the lesson.

FT-14 Offline playback

Control: Manual.

Initial State: All acceptance suite lessons are on the device and the device is in airplane mode.

Input: Open and play each lesson.

Output: Every lesson plays without a network connection.

Test Case Derivation: Fit criterion of RB2.

How test will be performed: Play each lesson on both devices in airplane mode.

4.1.3 Lesson Display and Content

These tests cover what the learner sees for each line and word (FR6, FR7, UH3, IN12, CU1, HS6, LG1, AU1).

FT-15 Display layers

Control: Manual.

Initial State: An acceptance suite lesson is open in each language.

Input: For every line, view the original script and stored segmentation, then turn the transliteration and the line translation on and off.

Output: The displayed words and readings match the song map, and each layer can be turned on and off.

Test Case Derivation: Fit criterion of FR6 (G5).

How test will be performed: A team member compares each line with

the map file, and a script checks the displayed word list against the stored words.

FT-16 Select a word or line

Control: Manual.
Initial State: A lesson is playing.
Input: Select every annotated target in the acceptance suite.
Output: The translation, reading, and annotation (if present) are shown and playback does not stop.
Test Case Derivation: Fit criterion of FR7 (G2).
How test will be performed: A team member selects each annotated target while the lesson plays and checks that the audio continues.

FT-17 Display settings persist

Control: Manual.
Initial State: Transliteration and translation are turned off.
Input: Close and reopen the app.
Output: The settings are unchanged.
Test Case Derivation: Fit criterion of UH3.
How test will be performed: Change the settings, close the app completely, and reopen it.

FT-18 Script rendering

Control: Manual.
Initial State: Each acceptance suite lesson is open on both devices.
Input: View every line.
Output: Every line renders with no boxes for missing glyphs, and the maps are read and written as UTF-8.
Test Case Derivation: Fit criterion of IN12.
How test will be performed: Compare screenshots of every line with the map text on iOS and Android.

FT-19 Hepburn romanization

Control: Manual.
Initial State: The Japanese acceptance suite lesson is open.
Input: View the transliteration of every word.
Output: Each word's romanization matches modified Hepburn romanization.
Test Case Derivation: Fit criterion of CU1.

How test will be performed: A team member checks every word against a Hepburn reference.

FT-20 Explicit content handling

Control: Manual.

Initial State: A test lesson with the explicit content flag set and an annotation for a vulgar term are loaded.

Input: Open the lesson with the hide-explicit setting off, then on. Also try to load a map without the flag.

Output: A warning is shown before the lesson opens, and the lesson is hidden when the setting is on. The map without the flag is rejected by the schema. The vulgar term's annotation includes a note on its register.

Test Case Derivation: Fit criterion of HS6.

How test will be performed: A team member runs the steps on both devices and checks the schema rejection with a Jest test.

FT-21 Credits shown

Control: Manual.

Initial State: Each acceptance suite lesson is open.

Input: Open the credits for each lesson.

Output: The source and licence information for the audio and lyrics is shown.

Test Case Derivation: Fit criterion of LG1, learner side.

How test will be performed: A team member compares the credits with the map's `source` fields.

FT-22 Error report

Control: Manual.

Initial State: A test lesson is open.

Input: Prepare a report for a lyric line, then complete it and, in a second attempt, cancel the share sheet.

Output: The report contains the map identifier, map version, line, and the learner's comment, and opens in the share sheet. When the share sheet is cancelled, nothing is sent by the app.

Test Case Derivation: Fit criterion of AU1.

How test will be performed: Run on both devices, and capture network traffic during the cancelled attempt.

4.1.4 Anki Handoff

These tests cover sending vocabulary to Anki (FR8, OE2, SL3, IN8 to IN10, AC1, AC2, PR2). They are run with the method selected by the PoC (risk R3). If more than one method is selected, they are run for each.

FT-23 Send a word to Anki

Control: Manual.
Initial State: The test deck is empty and Anki is available.
Input: Select one word in each acceptance suite language and send it to Anki.
Output: A note appears in the test deck with the *Term*, *Translation*, and *Context* fields containing the word, its translation, and the line it appears in.
Test Case Derivation: Fit criteria of FR8 and OE2 (G6).
How test will be performed: A tester sends the word and inspects the note in Anki.

FT-24 Anki unavailable

Control: Manual.
Initial State: Anki is not available.
Input: Send a word, wait, close and reopen the app, then make Anki available and retry.
Output: Within `ANKI_TIMEOUT` the app reports that sending failed and the item is queued. It is not reported as sent. The item is still queued after the restart and is delivered on retry. The learner can also export the queue to a file.
Test Case Derivation: Fit criteria of IN8 and SL3.
How test will be performed: A tester times the failure message and checks the queue after the restart. The test is also run with Anki available to check that the result is reported within `ANKI_TIMEOUT`.

FT-25 Duplicate send

Control: Manual.
Initial State: The test deck is empty.
Input: Send the same word twice, once by tapping the send control twice in quick succession.
Output: One note is created, and the send control is disabled while a request is in progress.

Test Case Derivation: Fit criterion of IN9.
How test will be performed: A tester sends the word twice and counts the notes in the test deck.

FT-26 Field names and encoding

Control: Manual.
Initial State: A note type that lacks the *Term*, *Translation*, or *Context* field, and a correct note type, are available.
Input: Send Japanese test words with each note type.
Output: With the correct note type, the Japanese text appears correctly in the right fields. With the incomplete note type, the app stops with a message and no note is created.
Test Case Derivation: Fit criterion of IN10.
How test will be performed: A tester inspects the notes in Anki after each send.

FT-27 Anki collection unchanged

Control: Automatic (integration test).
Initial State: A test Anki collection with existing decks, notes, and note types.
Input: Send several words.
Output: The test collection is unchanged apart from the added notes in the LingoLyrics deck.
Test Case Derivation: Fit criterion of AC1.
How test will be performed: Compare the collection before and after the test. Every Anki call is also reviewed (ST-4).

FT-28 AnkiConnect protection

Control: Manual. Applies only if AnkiConnect is used.
Initial State: AnkiConnect is configured as the setup instructions say.
Input: Send a request without the API key. Also inspect the allowed origin.
Output: The request is refused, and the allowed origin is the specific origin the product requires, never a wildcard.
Test Case Derivation: Fit criterion of AC2.
How test will be performed: Send the request from a script and read the configuration. The instructions are also reviewed (ST-3).

FT-29 Permission request

Control: Manual. Applies on Android if the AnkiDroid API is used.
Initial State: The app is freshly installed.
Input: Use the app without sending a word, then send the first word.
Output: The AnkiDroid permission is requested only at the first send, with an explanation, and no other sensitive permission is requested.
Test Case Derivation: Fit criterion of PR2.
How test will be performed: A tester follows the prompt sequence and reviews the app manifest (ST-5).

4.1.5 Editor

These tests cover the editor use case PUC4 and the editor's safety and security requirements (FR9 to FR14, HS9, IN4 to IN7, LG1, AU2, OE3).

FT-30 Load material and reopen a map

Control: Manual.
Initial State: The editor is open with no project.
Input: Load the test track's audio and lyrics. Populate the required timings, verified segmentation, annotations, metadata, and licence information using the supplied test material. Confirm that validation passes, export the map, close the editor, and reopen the exported map.
Output: The audio and lyrics load, and the reopened map has no data lost.
Test Case Derivation: Fit criterion of FR9.
How test will be performed: A tester compares the reopened map with the exported map field by field using a script.

FT-31 Timing, latency offset, and preview

Control: Manual (run), Automatic (analysis).
Initial State: The test track is loaded, and a latency offset has been set by tapping along to the test click track.
Input: Mark the start time of each line while the audio plays, adjust times by hand, and play the preview. Also play the same map in the learner app.
Output: With the offset applied, the preview's line changes meet the pass rate of FT-8, and the preview's line changes match the learner app's within `SYNC_TOL`.
Test Case Derivation: Fit criteria of FR10, FR13, and IN7.
How test will be performed: Log the highlight times in the preview

and in the app and compare them. Run once with wired and once with Bluetooth headphones.

FT-32 Segmentation suggestions

Control: Manual (review), Automatic (comparison).
Initial State: The Japanese test song is loaded in the editor.
Input: Request suggested segmentation and readings for `SEG_SAMPLE` test lines.
Output: At least `SEG_ACC` of the suggested word boundaries match a segmentation checked by the team. Suggestions are shown for review and can be accepted or corrected. Together with NT-1, this is the R2 PoC check.
Test Case Derivation: Fit criteria of FR11 and PA3, and the R2 pass criterion in the Development Plan.
How test will be performed: The team prepares and checks a reference segmentation before the test. A script computes the percentage of matching boundaries.

FT-33 Enter lesson content

Control: Manual.
Initial State: The test track is loaded.
Input: Enter every field of the data dictionary: line and word translations, annotations, and the lesson metadata.
Output: Each field can be entered and appears in the exported map.
Test Case Derivation: Fit criterion of FR12.
How test will be performed: Compare the exported map with the entered values.

FT-34 Export gating

Control: Automatic (rules), Manual (editor).
Initial State: A valid map is open in the editor.
Input: Try to export (a) each map in the invalid map set, (b) a map with an unverified segmentation line, (c) a map with empty licence fields, and (d) try to mark as published a map with an annotation that has no reviewer or whose reviewer is its author.
Output: Export is refused for (a) to (c), and the rules that fail are listed and match the rules the map breaks. (d) cannot be marked published. A valid map with verified lines, licence fields, and reviewed annotations exports.

Test Case Derivation: Fit criteria of FR14, IN4, IN5, and LG1, and the definitions of valid and publishable in the SRS.
How test will be performed: Run the maps through the editor's export and compare the listed rules with the expected rules.

FT-35 Loudness measurement and warning

Control: Manual (editor), Automatic (comparison).
Initial State: Test tracks with known integrated loudness are available, including one outside `TARGET_LOUDNESS` $\pm$ `LOUDNESS_TOL`.
Input: Load each track, then try to export.
Output: The displayed loudness is within 1 LU of the known value and is stored in the map's `loudness` field. The track outside the tolerance produces a warning before export.
Test Case Derivation: Fit criterion of HS9.
How test will be performed: Compare the displayed and stored values with the known values.

FT-36 Autosave, restore, undo, and confirmation

Control: Manual.
Initial State: A draft is open in the editor.
Input: Make an edit, wait at least `AUTOSAVE_INT`, close the tab, and reopen the editor. Use undo and redo for several edits, and perform an operation that changes more than one line. Also try to close with unsaved changes.
Output: The draft is restored. Undo and redo work for all edits. A warning appears before closing with unsaved changes, and the multiple-line operation asks for confirmation.
Test Case Derivation: Fit criterion of IN6.
How test will be performed: Record the time of the edit and inspect the persisted draft while the tab remains open, without manually saving. Confirm that the edit is persisted within `AUTOSAVE_INT`. Then close and reopen the editor, accept the restore option, and compare the restored draft with the saved state. For undo and redo, record the state before and after each tested edit and compare the resulting states.

FT-37 Export history

Control: Automatic.
Initial State: A valid map is open.
Input: Export the map twice.

Output: The map's `exports` list has two entries, each with the date, the creator, and the map version.
Test Case Derivation: Fit criterion of AU2.
How test will be performed: Jest test on the export function.

FT-38 Audio format on both platforms
Control: Manual.
Initial State: Each acceptance suite package has been exported by the editor.
Input: Open and play each package on an iOS and an Android device.
Output: Every package plays on both platforms, and the editor exports only audio formats that the learner app plays on both.
Test Case Derivation: Fit criterion of OE3 (G3).
How test will be performed: A tester plays each package on both devices.

4.2 Tests for Nonfunctional Requirements

The nonfunctional requirements for accuracy (PA2, PA3) are tested by FT-8 and FT-32, which report the percentage of matching results and the relative error. The tests below cover speed, accessibility, capacity, compatibility, installability, privacy, and usability. Reviews and inspections that do not follow the test format are in Section 4.2.6.

4.2.1 Performance

NT-1 Lesson load time
Type: Dynamic, Manual, performance.
Initial State: The test phone has the acceptance suite lessons and the app is closed.
Input/Condition: Open each lesson.
Output/Result: Each lesson is ready to play within `LOAD_TIME`. The measured times are reported for each lesson.
How test will be performed: Perform five runs per acceptance-suite lesson on each test device, fully closing and relaunching the app between runs. Start timing when the learner selects the lesson. Stop when its audio is loaded, its initial lyrics are displayed, and playback is enabled. Record the device model, operating system, build commit, and all five

durations. Report the minimum, median, and maximum. Every measured run must meet `LOAD_TIME`.
Requirements: SL1 (also the R2 PoC criterion).

NT-2 Selection response time
Type: Dynamic, Manual, performance.
Initial State: A lesson is playing on the test phone.
Input/Condition: Select annotated words and lines.
Output/Result: The translation and annotation appear within `SELECT_RESPONSE` of each selection.
How test will be performed: Instrumented timestamps record the time from selection to display.
Requirements: SL2.

4.2.2 Accessibility and Appearance

NT-3 Flashing and reduced motion
Type: Dynamic, Manual and Automatic.
Initial State: The fastest acceptance suite song is open, first with the reduce motion setting off and then on.
Input/Condition: Play the song.
Output/Result: The highlighting produces no more than `MAX_FLASHES` large area flashes in any period of one second, and the app follows the reduce motion setting.
How test will be performed: Count the highlight changes per second from the position log, and check the behaviour with reduce motion turned on.
Requirements: HS4, ST1.

NT-4 Contrast
Type: Static analysis, Manual.
Initial State: The release build is installed.
Input/Condition: Every text colour and highlight colour pair used in the learner app.
Output/Result: Each pair meets WCAG 2.1 level AA contrast.
How test will be performed: Check each pair with a contrast checker.
Requirements: ST1.

NT-5 Non-colour cue, layout, and assistive technology

Type: Manual.
Initial State: The release build is installed, with large system text and VoiceOver or TalkBack available.
Input/Condition: (a) Take greyscale and colour blindness simulations of a playing lesson. (b) Take screenshots of the lesson view on a phone screen. (c) Use the largest system text size. (d) Operate every control with VoiceOver on iOS and TalkBack on Android.
Output/Result: (a) The active line and word can be identified. (b) The lyric area takes up more of the screen than the playback controls and menus. (c) The lyrics remain readable and scale with the text size setting. (d) Each control is announced.
How test will be performed: A team member carries out the steps on both devices and records the screenshots.
Requirements: LF1, LF2, HS5, UH7.

4.2.3 Capacity

NT-6 Capacity

Type: Dynamic, Manual.
Initial State: The test phone has no lessons.
Input/Condition: (a) A test map at `MAX_AUDIO_LENGTH` and `MAX_LESSON_LINES`. (b) Import `LIBRARY_SIZE` lessons.
Output/Result: (a) The map opens in the app and the editor, plays, and meets the pass rate of FT-8. (b) All lessons are imported and listed.
How test will be performed: Run FT-8 on the limit map, then import the lessons and view the list.
Requirements: CP1, CP2.

4.2.4 Compatibility, Installability, and Privacy

NT-7 Schema version compatibility

Type: Automatic.
Initial State: CI has example maps for every released minor version of the schema.
Input/Condition: Open each map with the current app.
Output/Result: Every valid map of a supported major version opens.
How test will be performed: A CI job opens each map with the current

app code.
Requirements: LO1.

NT-8 Installation and platforms

Type: Manual.
Initial State: One iOS device, one Android device, and each supported browser.
Input/Condition: Install the Expo development or preview build of the learner app on each device. Open the editor from its URL in each supported browser.
Output/Result: The same release build runs on both platforms from one code base, and the editor runs with no installation.
How test will be performed: A team member follows the installation steps in the user documentation on a clean device.
Requirements: OE4, MS4.

NT-9 No other outbound data

Type: Dynamic, Manual.
Initial State: A test session on the release build, with network capture running.
Input/Condition: Open and play lessons, send words to Anki, and prepare an error report that is cancelled.
Output/Result: The only outbound traffic is the notes sent to Anki. The app sends nothing when the share sheet is cancelled.
How test will be performed: Review the network capture together with a review of the dependencies.
Requirements: PR1, AU1.

4.2.5 Usability Studies

Both studies are part of the Usability Report. The tasks are in Appendix 6.2.

NT-10 Creator study

Type: Manual, usability.
Initial State: Three test creators receive the same supplied audio, lyric, and annotation material for the test track (at most `TEST_TRACK_LINES` lines), the creator guide, and no other help.
Input/Condition: Each creator produces a complete map with the editor.

Output/Result: At least `CREATE_PASS` creators finish within `CREATE_TIME` without editing the map file by hand, and each resulting map validates and plays in the learner app. The completion time and any difficulties are recorded.
How test will be performed: A team member observes each session and times it. The result map is checked with the validator and played in the app.
Requirements: UH2, FR10, FR15, MS3 (G4).

NT-11 Learner study

Type: Manual, usability.
Initial State: Three representative learners who are not project team members use the release build for the first time, with only the written learner instructions.
Input/Condition: Each learner opens a lesson, plays it, displays a word's annotation, and sends the word to Anki. Taps from the home screen to playback are counted, and the controls are used with one hand.
Output/Result: At least `CREATE_PASS` of the participants complete the tasks within `LEARN_TIME` without help. Playback starts within `MAX_TAPS_PLAY` taps from the home screen. The participants operate the controls without instruction, and all lesson controls can be reached with one hand.
How test will be performed: A team member observes each session and records the times, the taps, and any difficulties. Timing starts when the participant begins the first task and ends when the final task is completed. Successful completion requires opening and playing the lesson, displaying the selected word's annotation, and confirming that the correct note appears in Anki. If assistance is provided, record it and do not count that session as an unassisted success.
Requirements: UH5, UH1, LF3, OE1, MS3.

4.2.6 Static Tests

The following requirements are verified by review, inspection, or inspection of CI output rather than by running the product. Each review is carried out by at least one team member other than the person who wrote the item being reviewed, and findings are recorded as issues.

Table 4: Static tests

ID	What is reviewed and how	Who	Requirements
ST-1	Every error message string in the app and the editor is read against the rule that it says what failed and what the user can do next, in plain language and without blame. All interface text is checked to be in English.	Content and QA Manager and one other member	UH6, UH4
ST-2	Design review of the release build: no streak loss penalties, countdown timers, or unsolicited push notifications, and each lesson's difficulty is shown before it is opened.	QA Manager and a member who does not own the learner app	HS7
ST-3	Review of the AnkiConnect setup instructions and default configuration for the specific origin, the API key requirement, and the warning before any step that opens AnkiConnect to another address.	Anki owner and a reviewer	AC2
ST-4	Code review of every Anki call to confirm that notes are only added, and only to the LingoLyrics deck.	Anki owner's backup	AC1

ID	What is reviewed and how	Who	Requirements
ST-5	Review of the app manifest and the permission prompt sequence.	Anki owner's backup	PR2
ST-6	Inspection of the CI configuration and run history: example maps are validated on each pull request, the coverage report shows at least `COVERAGE` for the shared core, lock files are committed, the vulnerability check runs, and every pull request that changes dependencies has a second approval.	DevOps manager and a reviewer	MS1, MS2, IM3
ST-7	Check of release tags and notes: each release is tagged on `main` and states the schema version it supports.	QA Manager	OE5
ST-8	The schema is validated against the JSON Schema Draft 2020-12 metaschema, maps are valid UTF-8 JSON, and the time fields are checked for a resolution of `TIME_RES` or finer.	Editor and schema owner and a reviewer	ST2, PA1

ID	What is reviewed and how	Who	Requirements
ST-9	Review of the acceptance suite annotations: annotations of culturally specific references explain the reference rather than only translate it, and each annotation has a reviewer other than its author.	Language processing owner and a second member	CU2, IN5
ST-10	Review that the learner and creator instructions exist and cover importing lessons, setting up Anki, using the editor, and the licence fields. Their use is checked in NT-10 and NT-11.	QA Manager	MS3
ST-11	Review when the design is complete: the learner app contains no tokenizer, the editor and app share one validation module, and language-specific schema fields are optional and keyed by language code.	Team review of the Module Guide	IN4, FR16, SC1

4.3 Traceability Between Test Cases and Requirements

Table 5 shows which tests support which requirements. Requirements are defined in the SRS, where each requirement also lists the goals, use cases, and hazards it traces to. The SRS Priority 1 requirements are tested first, in the PoC.

Table 5: Tests and the requirements they support

Test	Requirements
FT-1	FR1, FR2
FT-2	FR1, FR14, IN2, FR16
FT-3	IN3, LO1
FT-4	IN11
FT-5	RB1, IM1, IM2
FT-6	FR16, MS1
FT-7	FR3
FT-8	FR4, FR5, PA2
FT-9	IN1
FT-10	HS1
FT-11	HS2
FT-12	HS8
FT-13	HS3
FT-14	RB2
FT-15	FR6
FT-16	FR7
FT-17	UH3
FT-18	IN12
FT-19	CU1
FT-20	HS6
FT-21	LG1
FT-22	AU1
FT-23	FR8, OE2
FT-24	IN8, SL3
FT-25	IN9
FT-26	IN10
FT-27	AC1
FT-28	AC2

Test	Requirements
FT-29	PR2
FT-30	FR9
FT-31	FR10, FR13, IN7
FT-32	FR11, PA3
FT-33	FR12
FT-34	FR14, IN4, IN5, LG1
FT-35	HS9
FT-36	IN6
FT-37	AU2
FT-38	OE3
NT-1	SL1
NT-2	SL2
NT-3	HS4, ST1
NT-4	ST1
NT-5	LF1, LF2, HS5, UH7
NT-6	CP1, CP2
NT-7	LO1
NT-8	OE4, MS4
NT-9	PR1, AU1
NT-10	UH2, FR10, FR15, MS3
NT-11	UH5, UH1, LF3, OE1, MS3
ST-1	UH6, UH4
ST-2	HS7
ST-3	AC2
ST-4	AC1
ST-5	PR2
ST-6	MS1, MS2, IM3
ST-7	OE5
ST-8	ST2, PA1
ST-9	CU2, IN5

Test	Requirements
ST-10	MS3
ST-11	IN4, FR16, SC1

The goals are covered as follows: G1 by FT-8, FT-9, FT-31, and NT-6; G2 by FT-16 and NT-2; G3 by FT-1, FT-2, FT-6, and FT-38; G4 by NT-10; G5 by FT-15, FT-18, FT-19, and FT-32; and G6 by FT-23 and FT-24.

5 Unit Test Description

This section will be completed after the Module Interface Specification is finished (Design Document Revision −1 is due November 16, 2026). It will refer to the MIS and explain how test cases are chosen for each module.

5.1 Unit Testing Scope

The following is already known. The shared core (the schema, the validator, and the shared types) has the highest priority because it decides which lessons can be opened, and it must have statement coverage of at least `COVERAGE` (MS2). The libraries from other vendors are not unit tested (Section 2). The remaining modules and their priorities will be listed once the Module Guide is complete.

5.2 Tests for Functional Requirements

Unit tests will use Jest and run in CI on every pull request. For the validator, there will be at least one invalid test map for each of the conditions V1 to V8 (IN2). The tests for each module will be written from black box and white box perspectives and listed here by module once the MIS exists.

5.3 Tests for Nonfunctional Requirements

Modules that need to be assessed independently for performance will be listed here once the design is complete. Performance data may also be taken from the system tests NT-1 and NT-2.

5.4 Traceability Between Test Cases and Modules

This table will be provided once the modules are defined. It will show that every module has been considered.

References

Team LingoLyrics. Software requirements specification for LingoLyrics: A language learning system based on songs. https://github.com/rhea333/LingoLyrics/blob/main/pdfs/SRS-Volere/SRS.pdf, October 2026. Created October 18, 2026.

6 Appendix

6.1 Symbolic Parameters

The definition of the test cases calls for symbolic constants. Their values are defined in Table 6 for easy maintenance and must stay consistent with the SRS appendix on symbolic constants. Values marked *initial* will be confirmed during the PoC or usability testing.

Table 6: Symbolic parameters used by the tests

Constant	Value	Used by
SYNC_TOL	0.5 s	FT-7, FT-8, FT-31
SYNC_PASS	95 %	FT-8
CREATE_TIME	45 min	NT-10
CREATE_PASS	2 of 3	NT-10, NT-11
TEST_TRACK_LINES	12 lines	NT-10
LANG_COUNT	2	Test environment
LOAD_TIME	3 s	NT-1
SEG_ACC	90 %	FT-32
SEG_SAMPLE	20 lines	FT-32
SELECT_RESPONSE	0.5 s (initial)	NT-2
ANKI_TIMEOUT	5 s (initial)	FT-24
TIME_RES	0.01 s	ST-8
AUDIO_DUR_TOL	1 s (initial)	FT-4
DISCONNECT_PAUSE	1 s	FT-11
MAX_FLASHES	3 per second	NT-3
FADE_IN	0.5 s (initial)	FT-12
TARGET_LOUDNESS	−16 LUFS (initial)	FT-35
LOUDNESS_TOL	2 LU (initial)	FT-35
AUTOSAVE_INT	60 s	FT-36
MAX_MAP_SIZE	5 MB, excluding audio (initial)	FT-5

Constant	Value	Used by
MAX_NESTING	32 levels (initial)	FT-5
SUPPORTED_MAJOR	{1}	FT-3
MAX_TAPS_PLAY	3 (initial)	NT-11
LEARN_TIME	10 min (initial)	NT-11
MAX_AUDIO_LENGTH	10 min (initial)	NT-6
MAX_LESSON_LINES	150 lines (initial)	NT-6
LIBRARY_SIZE	20 lessons (initial)	NT-6
COVERAGE	80 % (initial)	ST-6

6.2 Usability Tasks and Survey Questions

The tasks below come from requirements UH2 and UH5. After completing the tasks, participants answer the questions below independently. Responses will help explain the observed difficulties and guide improvements.

Creator study (NT-10). Using the creator guide and the supplied material for the test track, the creator:

1. loads the audio and lyric text;
2. marks the start time of each line while the audio plays and adjusts times as needed;
3. reviews the suggested segmentation and readings, if the language needs them;
4. enters the translations, annotations, and the source and licence information; and
5. previews the lesson and exports the song map.

Learner study (NT-11). Using the learner instructions, the learner:

1. opens a lesson that has been imported;

2. plays the lesson;

3. displays the annotation of a word; and

4. sends the word to Anki.

Post-task questions. Participants rate each applicable statement from 1 (strongly disagree) to 5 (strongly agree), with a "not applicable" option.

Creators:

1. It was easy to mark and adjust lyric timings.

2. It was easy to review and correct suggested word segmentation.

3. The preview helped me identify mistakes in my lesson.

4. Validation messages explained what I needed to fix before export.

Learners:

1. It was easy to open a lesson and start playback.

2. It was easy to follow the highlighted lyrics.

3. It was easy to find a selected word's meaning.

4. It was clear whether my selected word had been sent to Anki.

Open-ended questions for both groups:

1. Which task was most difficult, and why?

2. Was any instruction, control, or message confusing?

3. What one change would make the product easier to use?

Responses will be summarized separately for creators and learners. Survey ratings supplement the observed completion times, errors, and task outcomes; they do not replace the pass criteria of NT-10 and NT-11.

7 Appendix — Generative AI Use Disclosure

7.1 AI Tools Used

Claude Sonnet 5.5 (Anthropic), used in the Claude chat interface (claude.ai).

7.2 How and Where Used

Claude was given the blank V&V Plan template together with the Problem Statement, the Development Plan, and the SRS, and was used to draft the text of Sections 1 to 3 and Appendix A and B of this document, including the tests, the static tests, and the traceability table. The tests were derived from the fit criteria of the SRS, and the constants and thresholds were taken from the SRS appendix.

7.3 Verification of AI-Generated Content

8 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?